True/False:
The function range() in Python returns a generator object that lazily evaluates its values.

True/False
Profiling is a static program analysis technique that allows developers to analyze code performance without running the program.
True/False:
The args attribute of an exception contains a tuple of all arguments passed to the exception during initialization.

True/False:
The json.dumps() method can be used to directly write a Python dictionary to a .json file.

True/False:
The expression range(0) and range(5, 0) both return empty sequences.

What design principle is most clearly violated when a class like Character directly handles all weapon attack logic?
A. Dependency Inversion Principle
B. Interface Segregation Principle
C. Single Responsibility Principle
D. Liskov Substitution Principle

In which of the following software designs would the Mediator pattern be the most appropriate to use?
A. A real-time multiplayer game server where all player objects directly update each other's positions and scores in shared memory.
B. A modular dashboard application where widgets (e.g., charts, filters, status panels) need to update one another's state based on user actions, but should remain independent and reusable.
C. A text processing utility where each function (e.g., tokenizer, stemmer, parser) passes data linearly to the next using a pipeline.
D. A filesystem monitoring tool where file watchers trigger logging operations using observer callbacks.

Which of the following best illustrates a disadvantage of the Factory Pattern in a large-scale system?
A. The use of constructors is strictly prohibited in client code.
B. The system becomes overly reliant on subclassing and inheritance.
C. The number of classes may grow quickly due to the need for one factory per product.
D. It cannot be used in combination with other design patterns.

You are developing a file storage service. You want to encrypt and compress files before

writing, and reverse the process during reading. Which pattern offers the most flexible solution?

A. Proxy
B. Facade
C. Bridge
D. Decorator

For the following code snippet, answer the question below:

```
from abc import ABC, abstractmethod

class TrafficLight(ABC):
    @abstractmethod
    def next(self, traffic_light):
        pass


class Red(TrafficLight):
    def next(self, traffic_light):
        print("Red light -> Switching to green.")
        traffic_light.change_light(Green())

class Yellow(TrafficLight):
    def next(self, traffic_light):
        print("Yellow light -> Switching to red.")
        traffic_light.change_light(Red())

class Green(TrafficLight):
    def next(self, traffic_light):
        print("Green light -> Switching to yellow.")
        traffic_light.change_light(Yellow())

# Context
class TrafficLight:
    def __init__(self):
        self.variable = Red()  # Initial is Red

    def change_light(self, light):
        self.variable = light

    def switch(self):
        self.variable.next(self)

# Usage
traffic_light = TrafficLight()

# Simulate traffic light changes
```

```
traffic_light.switch()  # Red -> Green
traffic_light.switch()  # Green -> Yellow
traffic_light.switch()  # Yellow -> Red
traffic_light.switch()  # Red -> Green (cycle repeats)
```

Which of the following statements best describes the design approach used in the TrafficLight implementation?

A. It demonstrates the Strategy pattern, allowing different algorithms to be chosen at runtime without altering the context.

B. It uses the Chain of Responsibility pattern to pass control between handlers based on traffic light color.

C. It implements the State pattern, encapsulating state-specific behavior and enabling dynamic transitions based on the current state.

D. It simulates asynchronous behavior using the Observer pattern to notify components about light changes.

Roughly how long will it take for the following code to execute?

```
import asyncio
import time


class Subject:
    def __init__(self):
        self._observers = []
        self.news_data = {
            "sports": ["Game 1 Highlights", "Player Transfers", "Match Schedules"],
            "tech": ["New AI Breakthrough", "Latest Gadgets", "Tech Company Earnings"]
        }

    def attach(self, observer):
        if observer not in self._observers:
            self._observers.append(observer)

    def detach(self, observer):
        if observer in self._observers:
            self._observers.remove(observer)

    async def notify(self, category):
        for observer in self._observers:
            await observer.update(self, category)

    async def get_latest_news(self, category):
        # Simulate a slight delay in fetching news data
        await asyncio.sleep(0.1)
        return self.news_data.get(category)
```

```
class Observer:
    async def update(self, subject, category):
        # Simulate time to receive the category update
        await asyncio.sleep(0.5)
        print(f"Observer received category update: {category}")
        news_items = await subject.get_latest_news(category)
        print(f"Fetching news in category '{category}': {news_items}")

class ObserverA(Observer):
    async def update(self, subject, category):
        if category == "sports":
            await super().update(subject, category)
            print(f"ObserverA processed category: {category}")

class ObserverB(Observer):
    async def update(self, subject, category):
        await super().update(subject, category)
        print(f"ObserverB processed category: {category}")

async def main():
    start_time = time.perf_counter()

    subject = Subject()
    observer_a = ObserverA()
    observer_b = ObserverB()

    subject.attach(observer_a)
    subject.attach(observer_b)

    task_notify1 = subject.notify("sports")
    subject.detach(observer_a)
    task_notify2 = subject.notify("tech")

    await asyncio.gather(task_notify1, task_notify2)

    end_time = time.perf_counter()
    print(f"Total time: {end_time - start_time:.2f} seconds")


if __name__ == "__main__":
    asyncio.run(main())
```

Answer: ~0.6 seconds

Write Code

Complete the class DigitSumIterator, which iterates over **positive integers whose digits sum up**

to a specified target. You must implement the __next__ and digit_sum methods so that it finds the next such number.

```
class DigitSumIterator:
    """
    An iterator that yields numbers whose digits sum up to a target value.
    """

    def __init__(self, target_sum, num_terms):
        """
        Initializes the iterator with the digit sum target and number of terms to return.
        :param target_sum: the desired digit sum (e.g., 10)
        :param num_terms: how many matching numbers to return
        """
        self.target_sum = target_sum
        self.num_terms = num_terms
        self.count = 0
        self.current = 1

    def __iter__(self):
        return self

    def digit_sum(self, n):
        """
        Returns the sum of the digits of n.
        """
        # TODO: Implement this method to calculate the digit sum of n.
        # Hint: Use the modulus operator and integer division to extract digits.

    def __next__(self):
        # Your code here
```

Example Usage

```
iterator = DigitSumIterator(target_sum=10, num_terms=5)
for num in iterator:
    print(num)
```

Expected Output: *(These are the first 5 numbers whose digits add up to 10)*

```
19
28
37
46
55
```

Solution

```
    def __next__(self):
        while self.count < self.num_terms:
```

```
        if self.digit_sum(self.current) == self.target_sum:
            result = self.current
            self.count += 1
            self.current += 1
            return result
        self.current += 1
    raise StopIteration
def digit_sum(self, n):
    """
    Returns the sum of the digits of n
    """
    total = 0
    while n > 0:
        total += n % 10
        n //= 10
    return total
```

Refactoring

You are building a notification system using the Decorator Pattern. The goal is to allow notifications to be extended with additional behaviors (e.g., email, SMS, Slack) without modifying the base Notifier class.

Assume you have the following code which is supposed to implement the Decorator Pattern, but it's not quite right. Your task is to fix it while keeping the client code unchanged.

```
class Notifier:
    def send(self):
        print("Sending basic notification")


class NotifierWithEmail(Notifier):
    def send(self):
        print("Sending basic notification")
        print("Sending email notification")


class NotifierWithSMS(Notifier):
    def send(self):
        print("Sending basic notification")
        print("Sending SMS notification")


class NotifierWithEmailAndSMS(Notifier):
    def send(self):
        print("Sending basic notification")
        print("Sending email notification")
        print("Sending SMS notification")
```

```
notifier1 = NotifierWithEmail()
notifier1.send()
# Output:
# Sending basic notification
# Sending email notification

notifier2 = NotifierWithEmailAndSMS()
notifier2.send()
# Output:
# Sending basic notification
# Sending email notification
# Sending SMS notification
```

** What's Wrong? **

This implementation violates the Decorator Pattern specification in several ways:

- EmailNotifier and SMSNotifier don't decorate an existing notifier. They completely override it.
- There's no composition. Just inheritance.

Your Task

- Fix the design so that you can dynamically stack notifications like:
- notifier = SMSNotifier(EmailNotifier(Notifier()))
  notifier.send()
- Output:
- Sending basic notification
  Sending email notification
  Sending SMS notification

don't modify the client code.

- Draw a UML class diagra of the corrected design.

Solution:

```
class Notifier:
    def send(self):
        print("Sending basic notification")


class NotifierDecorator(Notifier):
    def __init__(self, notifier):
        self._notifier = notifier

    def send(self):
        self._notifier.send()


class EmailNotifier(NotifierDecorator):
    def send(self):
        super().send()
```

```
        print("Sending email notification")


class SMSNotifier(NotifierDecorator):
    def send(self):
        super().send()
        print("Sending SMS notification")


notifier = SMSNotifier(EmailNotifier(Notifier()))
notifier.send()
```

UML:

TRUE/FALSE (5 Marks. Half Mark for Each Question)

❌Q1. In Python, it's possible to modify the value of a tuple element after it has been created.

✔Q2. Everything in Python is an object.

✔Q3. The Dependency Inversion Principle (DIP) suggests that both high-level and low-level modules should depend on abstractions, not concrete implementations, which promotes loose coupling and flexibility in the codebase.

❌Q4. The output of the following code snippet is

```
x = [[i * j for j in range(1, 2)] for i in range(1, 3)]
print(x)
```

[[1, 2]]

Feedback output

[[1], [2]]

✔Q5. The output of the following

```
class Parent:
    def __init__(self, name):
        self.name = name

    def greet(self):
        return f"Hello, I am {self.name}."


class Child(Parent):
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def greet(self):
        return f"{super().greet()} I am {self.age} years old."


child = Child("Alice", 10)

print(child.greet())
```

is

Hello, I am Alice. I am 10 years old.

✔Q6. Decorator and Proxy have similar structures, but very different intents.

❌Q7. In the State Design Pattern, state objects are completely independent and unaware of each other.

Feedback- that is the observer design pattern

✔Q8. Facade and Mediator have similar jobs: they try to organize collaboration between lots of tightly coupled classes.

For the following code, answer the following two questions.

```
def safe_divide(x, y):
    try:
        return x / y
    except ZeroDivisionError:
        return None  # Or you might want to raise a custom exception

import unittest


class TestExceptionHandling(unittest.TestCase):
    def test_divide_by_zero(self):
        with self.assertRaises(ZeroDivisionError):
            safe_divide(5, 0)

    def test_safe_division(self):
        result = safe_divide(10, 2)
        self.assertIsNone(result)  # Assuming you return None on error

# Run the tests
unittest.main()
```

❌Q9 - The test_divide_by_zero unit test will pass.

❌Q10 - The test_safe_division unit test will pass.

MCQ (10 Marks. One Mark for Each Question)

Q1. Which of the following design patterns construct complex objects step by step.
A. Builder
B. Abstract Factory
C. Singleton
D. Factory

Q2. Which of the following design patterns address various ways of connecting senders and receivers of requests?

B. Chain of Responsibility, Observer and Strategy
C. Chain of Responsibility, Mediator and Strategy
C. Chain of Responsibility, Mediator and Observer
D. Mediator, Observer and Strategy

Q3. What is the output of the following code?

```
class SupportHandler:
    """
    The base abstract handler for the chain of responsibility.
```

```
    """
    def __init__(self, successor=None):
        self._successor = successor

    def handle_request(self, request):
        """
        Handles the request if possible. Otherwise passes it to the successor.
        """
        if self._can_handle(request):
            self._process_request(request)
        elif self._successor is not None:
            print(f"{self.__class__.__name__} cannot handle. Passing to next handler...")
            self._successor.handle_request(request)
        else:
            print("Request cannot be handled!")

    def _can_handle(self, request):
        raise NotImplementedError("Subclasses must implement this method")

    def _process_request(self, request):
        raise NotImplementedError("Subclasses must implement this method")


class Level1Support(SupportHandler):
    def _can_handle(self, request):
        return request.severity == "low"

    def _process_request(self, request):
        print(f"Level 1 Support: Handling request - {request}")


class Level2Support(SupportHandler):
    def _can_handle(self, request):
        return request.severity == "medium"

    def _process_request(self, request):
        print(f"Level 2 Support: Handling request - {request}")


class Level3Support(SupportHandler):
    def _can_handle(self, request):
        return request.severity == "high"

    def _process_request(self, request):
        print(f"Level 3 Support: Handling request - {request}")


class Request:
    """A simple request class"""
    def __init__(self, description, severity):
        self.description = description
        self.severity = severity

    def __str__(self):
        return f"Request({self.description}, severity={self.severity})"


# Building the chain of responsibility
level1 = Level1Support()
level2 = Level2Support(level1)
level3 = Level3Support(level2)

# Sample requests
requests = [
    Request("Basic printer issue", "low")
]
```

```
# Process the requests through the chain
for request in requests:
    level3.handle_request(request)  # Start with the highest level
```

Option A.

Level3Support cannot handle. Passing to next handler...
Level2Support cannot handle. Passing to next handler...
Level 1 Support: Handling request - Request(Basic printer issue, severity=low)

Option B.

Level 1 Support: Handling request - Request(Basic printer issue, severity=low)
Level2Support cannot handle. Passing to next handler...
Level3Support cannot handle. Passing to next handler...

Option C.

Level3Support cannot handle. Passing to next handler...
Level 2 Support: Handling request - Request(Basic printer issue, severity=low)

Option D.

Level1Support cannot handle. Passing to next handler...
Level2Support cannot handle. Passing to next handler...
Level 3 Support: Handling request - Request(Basic printer issue, severity=low)

Q4. Which design pattern is used in the following Duck simulator?

```
class Duck:
    def __init__(self, fly_behavior=None, quack_behavior=None):
        self._fly_behavior = fly_behavior
        self._quack_behavior = quack_behavior

    def perform_fly(self):
        self._fly_behavior.fly()

    def perform_quack(self):
        self._quack_behavior.quack()

    def swim(self):
        print("All ducks float, even decoys!")

    def display(self):
        """Each duck type will override this method to look different"""
        raise NotImplementedError


class FlyBehavior:
    """Parent class for fly behaviors."""

    def fly(self):
        raise NotImplementedError


class FlyWithWings(FlyBehavior):
    def fly(self):
        print("I'm flying with wings!!")
```

```
class FlyNoWay(FlyBehavior):
    def fly(self):
        print("I can't fly :(")



class QuackBehavior:
    """Parent class for quack behaviors"""

    def quack(self):
        raise NotImplementedError


class Quack(QuackBehavior):
    def quack(self):
        print("Quack!")


class Squeak(QuackBehavior):
    def quack(self):
        print("Squeak...")


class MuteQuack(QuackBehavior):
    def quack(self):
        print("<< Silence >>")


class MallardDuck(Duck):
    def display(self):
        print("I'm a real Mallard duck")


class RubberDuck(Duck):
    def display(self):
        print("I'm a rubber duckie")


# Client code in action
mallard = MallardDuck(FlyWithWings(), Quack())
mallard.display()
mallard.perform_fly()  # This duck can fly!
mallard.perform_quack()

rubber_duckie = RubberDuck(FlyNoWay(), Squeak())
rubber_duckie.display()
rubber_duckie.perform_fly()  # This duck cannot fly :(
rubber_duckie.perform_quack()
```

Option A. - The Observer design pattern
Option B. - The Bridge design pattern
Option C. - The state design pattern
Option D. - The Strategy design pattern

Q5. Which of the following UML diagrams is the correct representation of the following code

snippet?

```
import time

class TrafficLight:
    def __init__(self, state) -> None:
        self.state = state

    def update(self):
        """
        Updates the state of the traffic light
        Red -> Green -> Yellow -> Red
        """
        self.state = self.state.update()

    def change_state(self):
        self.state.set_context(self)
        self.state.update()


class State:
    def set_context(self, a_context):
        self.context = a_context


class Red(State):
    def __str__(self) -> str:
        return "Red"

    def update(self):
        self.context.state = Green()


class Green(State):
    def __str__(self) -> str:
        return "Green"

    def update(self):
        self.context.state = Yellow()


class Yellow(State):
    def __str__(self) -> str:
        return "Yellow"

    def update(self):
        self.context.state = Red()


def main():
    a_traffic_light = TrafficLight(Red())
    print(a_traffic_light.state)

    while True:
        print("Updating the state of the traffic light")
        a_traffic_light.change_state()
        print(a_traffic_light.state)
        time.sleep(1)  # import time
```

main()
Use the following key to identify the correct UML diagram:

- Option A
- Option B
- Option C

Q6. What is the output of the following code?

```
def data_source():
    for i in range(10):
        yield i


def f1(numbers):
    for num in numbers:
        yield num * 2


def f2(numbers):
    for num in numbers:
        if num % 2 == 0:
            yield num


# Pipeline execution
pipeline = f2(f1(data_source()))
for result in pipeline:
    print(result)
```

Option A:
0
4
8
12
16
Option B:
0
2
4

6
8
10
12
14
16
18
Option C:
0
2
4
6
8
10
12
14
16
18
20
Option D:
0
4
8
12
16
20

Q7. What is roughly the total time of running the following code?

```
import asyncio
import time


class Subject:
    def __init__(self):
        self._observers = []

    def attach(self, observer):
        if observer not in self._observers:
            self._observers.append(observer)

    def detach(self, observer):
        if observer in self._observers:
            self._observers.remove(observer)

    async def notify(self, message):
        for observer in self._observers:
            await observer.update(message)
```

```
class Observer:
    async def update(self, message):
        await asyncio.sleep(0.5)
        print(f"Observer received: {message}")


async def main():
    x = time.perf_counter()

    subject = Subject()
    observer_a = Observer()
    observer_b = Observer()

    subject.attach(observer_a)
    subject.attach(observer_b)

    task_notofiy = asyncio.create_task(subject.notify("Hello observers!"))

    another_task_notfiy = asyncio.create_task(subject.notify("Another message"))

    await task_notofiy
    await another_task_notfiy

    y = time.perf_counter()
    print(f"Total time: {y-x}")


if __name__ == "__main__":
    asyncio.run(main())
```

Option A - 1 second

Option B - .5 seconds

Option C - 2 seconds

Q8. Which of the following code snippets is a proper implementation of the Proxy design pattern?

Option A.

```
class SensitiveDatabase:
    def __init__(self):
        self._data = "Top Secret Data"

    def read_data(self):
        print("Accessing actual database...")
        return self._data


class DatabaseProxy:
    def __init__(self):
        self._database = SensitiveDatabase()
        self._authorized_users = ["admin", "manager"]

    def read_data(self, username):
        return self._database.read_data()
```

```
# driver code
database = DatabaseProxy()
print(database.read_data("admin"))
```

Option B.

```
class SensitiveDatabase:
    def __init__(self):
        self._data = "Top Secret Data"

    def read_data(self):
        print("Accessing actual database...")
        return self._data


class DatabaseProxy:
    def __init__(self):
        self._database = SensitiveDatabase()
        self._authorized_users = ["admin", "manager"]

    def read_data(self, username):
        if username in self._authorized_users:
            return self._database.read_data()
        else:
            print("Access denied!")

# driver code
database = DatabaseProxy()
print(database.read_data("admin"))
```

Option C.

```
class SensitiveDatabase:
    def __init__(self):
        self._data = "Top Secret Data"

    def read_data(self):
        print("Accessing actual database...")
        return self._data


class DatabaseProxy:
    def __init__(self):
        self._database = SensitiveDatabase()
        self._authorized_users = ["admin", "manager"]

    def read_data(self, username):
        if username in self._authorized_users:
            return self._database.read_data()
        else:
            print("Access denied!")
```

```
    def get_database(self):
        return self._database


# driver code
database = DatabaseProxy()
print(database.read_data("admin"))
```

Feedback

- Option A is wrong.

Bypassing the Proxy: The DatabaseProxy class should not provide a way to directly access the SensitiveDatabase. The proxy should be the only way to access the sensitive data, and it should control the access based on the user's credentials.

- Option C is wrong.

Also Bypassing the Proxy: because of the usage of ungaurded getter get_database() .

Q9. In Bridge design pattern, we say that the Abstractions control the Implementation. For the following code snippet, which class is the Abstraction and which class is the Implementation?

```
class Shape:
    def draw(self):
        pass


class Renderer:
    def render_circle(self, radius):
        pass


class Circle(Shape):
    def __init__(self, x, y, radius, renderer):
        self.x = x
        self.y = y
        self.radius = radius
        self.renderer = renderer

    def draw(self):
        self.renderer.render_circle(self.radius)


# ... Similar code for other shapes (Square, Triangle, etc.)


class RasterRenderer(Renderer):
    def render_circle(self, radius):
        print("Drawing a raster circle")


class VectorRenderer(Renderer):
    def render_circle(self, radius):
```

```
    print("Drawing a vector circle")

raster_circle = Circle(0, 0, 5, RasterRenderer())
raster_circle.draw()  # Output: Drawing a raster circle
```

Option A.

- Abstraction: Shape, Implementation: Renderer

Option B.

- Abstraction: Renderer, Implementation: Shape

Feedback

In the Bridge design pattern, the Abstraction class controls the Implementation class. In the given code snippet, the Shape class is the Abstraction class, and the Renderer class is the Implementation class. The Shape class defines the draw method, which is implemented by the Renderer class. The Circle class is a Shape subclass that uses the Renderer class to render the circle. The RasterRenderer and VectorRenderer classes are concrete implementations of the Renderer class, and the Circle class uses them to render the circle.

Q10. For the following UML, which design pattern is being implemented?

Option A. - The Observer design pattern
Option B. - The Strategy design pattern
Option C. - The Decorator design pattern
Option D. - The state design pattern
Feedback
This is an example from the slides. I just drew the UML.

What is the Output? (3 Marks)
What is a probable (likely) output of the following code?

```
import logging
import concurrent.futures
import time


class FakeDatabase:
    def __init__(self):
        self.value = 0

    def update(self, name):
        print("Thread %s: starting update"% name)
        local_copy = self.value
        local_copy += 1
        time.sleep(0.1)
        self.value = local_copy
        print("Thread %s: finishing update"% name)


if __name__ == "__main__":
    database = FakeDatabase()
    print("Testing update. Starting value is %d."% database.value)
    with concurrent.futures.ThreadPoolExecutor(max_workers=2) as executor:
        for index in range(2):
```

```
        executor.submit(database.update, index)
    print("Testing update. Ending value is %d."% database.value)
```

Answer

Testing update. Starting value is 0.
Thread 0: starting update
Thread 1: starting update
Thread 1: finishing update
Thread 0: finishing update
Testing update. Ending value is 1.

or

Testing update. Starting value is 0.
Thread 0: starting update
Thread 1: starting update
Thread 0: finishing update
Thread 1: finishing update
Testing update. Ending value is 1.

Short Written Questions (5 Marks)

Q1. (3 Marks) Complete the Intent and Use When columns of the following table for the Strategy, State, and Bridge design patterns. Use single short sentences to complete the Intent and Use When columns.

Summary Table

Design Pattern	Intent	Use When
Strategy		
State		
Bridge		

Answer

Summary Table

Design Pattern	Intent	Use When
Strategy	Select algorithms at runtime	Behavior needs to be selected/changed dynamically
State	Model state-dependent behavior	Object's behavior tightly tied to its internal state
Bridge	Separate abstraction from implementation for independent variation	Multiple implementations or platforms to work with

Q2. (2 Marks) Write a comprehension to generate the following output.

[(5, 'apple'), (6, 'banana'), (6, 'orange'), (4, 'kiwi'), (9, 'pineapple')]

from the following list

words = ["apple", "banana", "orange", "kiwi", "pineapple"]

Anwser

```
# Generating a sequence of tuples containing the lengths of words and the words themselves
words = ["apple", "banana", "orange", "kiwi", "pineapple"]
word_lengths = [(len(word), word) for word in words]
print(word_lengths)
```

Guided Written Question (6 Marks)

Rewrite the following code using the decorator pattern using the suggested UML diagram. The code defines a system for representing beverages with customizable options (condiments) and associated costs. It allows you to create different base beverages (like DarkRoast or HouseBlend) and add condiments with the ability to override default condiment prices.

```
class Beverage:
    def __init__(self):
        self.description = "Unknown Beverage"
        self.base_cost = 0.0

        self.has_milk = False
        self.has_mocha = False
        self.has_soy = False
        self.has_whip = False
        self.condiments_cost_table = {
            "milk": 0.10,  # Default costs
            "mocha": 0.20,
            "soy": 0.15,
            "whip": 0.10,
        }

    def get_description(self):
        desc = self.description
        if self.has_milk:
            desc += ", Milk"
        if self.has_mocha:
            desc += ", Mocha"
        if self.has_soy:
            desc += ", Soy"
        if self.has_whip:
            desc += ", Whip"
        return desc

    def cost(self, **kwargs):
        cost = self.base_cost

        if self.has_milk:
            cost += self.condiments_cost_table["milk"]
        if self.has_mocha:
            cost += self.condiments_cost_table["mocha"]
        if self.has_soy:
            cost += self.condiments_cost_table["soy"]
        if self.has_whip:
            cost += self.condiments_cost_table["whip"]

        # Override default costs with custom costs
        for condiment, custom_cost in kwargs.items():
```

```
            cost += custom_cost
            cost -= self.condiments_cost_table[condiment]
        return cost


class DarkRoast(Beverage):
    def __init__(self):
        super().__init__()
        self.description = "Dark Roast Coffee"
        self.base_cost = 0.99


class HouseBlend(Beverage):
    def __init__(self):
        super().__init__()
        self.description = "House Blend Coffee"
        self.base_cost = 0.89


# Example usage
beverage = DarkRoast()
beverage.has_mocha = True
beverage.has_whip = True
beverage.has_soy = True

print(beverage.get_description()) # Output: Dark Roast Coffee, Mocha, Whip, Soy
print(beverage.cost(soy=.12)) # Output: 1.41 (0.99 base + 0.20 mocha + 0.10 whip + 0.12 soy)
```

Here is a suggested UML diagram for the decorator pattern.

Step 1 – (1 Mark) Create the Beverage class.

```
class Beverage(ABC):
    """Abstract base class for beverages"""

    def __init__(self):
        self.description = "Unknown Beverage"

    def get_description(self):
        return self.description

    @abstractmethod
    def cost(self):
        pass
```

```
    @abstractmethod
    def cost(self):
        pass
    @abstractmethod
    def cost(self):
        pass
class Beverage(ABC):
    """Abstract base class for beverages"""
class Beverage(ABC):
    """Abstract base class for beverages"""
    def get_description(self):
        return self.description
    def get_description(self):
        return self.description
    def __init__(self):
        self.description = "Unknown Beverage"
    def __init__(self):
        self.description = "Unknown Beverage"
```

```
class HouseBlend(Beverage):
class HouseBlend(Beverage):

    def cost(self):
        return  0.89
    def cost(self):
        return  0.89
    def __init__(self):
        super().__init__()
        self.description = "Dark Roast Coffee"

    def __init__(self):
        super().__init__()
        self.description = "Dark Roast Coffee"
```

```
class DarkRoast(Beverage):
class DarkRoast(Beverage):

    def __init__(self):
        super().__init__()
        self.description = "House Blend Coffee"
    def __init__(self):
```

```
        super().__init__()
        self.description = "House Blend Coffee"
    def cost(self):
        return  0.99
    def cost(self):
        return  0.99
```

Step 3 – (1 Mark) Create the CondimentDecorator class

```
class CondimentDecorator(Beverage):
    """
    Base class for condiment decorators (with argument)
    """

    def __init__(self, beverage, condiment_cost):
        super().__init__()
        self.beverage = beverage
        self.condiment_cost = condiment_cost

    @abstractmethod
    def cost(self):
        pass
```

```
class CondimentDecorator(Beverage):
    """
    Base class for condiment decorators (with argument)
    """
class CondimentDecorator(Beverage):
    """
    Base class for condiment decorators (with argument)
    """
    def __init__(self, beverage, condiment_cost):
        super().__init__()
        self.beverage = beverage
        self.condiment_cost = condiment_cost
    def __init__(self, beverage, condiment_cost):
        super().__init__()
        self.beverage = beverage
        self.condiment_cost = condiment_cost
    @abstractmethod
    def cost(self):
```

```
        pass
    @abstractmethod
    def cost(self):
        pass
```

Step 4 – (2 Marks) Create the concrete decorator classes.

```
class Milk(CondimentDecorator):
    def __init__(self, beverage, custom_cost=0.10):
        super().__init__(beverage, custom_cost)

    def get_description(self):
        return self.beverage.get_description() + ", " + "Milk"

    def cost(self):
        return self.beverage.cost() + self.condiment_cost



class Mocha(CondimentDecorator):
    def __init__(self, beverage, custom_cost=0.20):
        super().__init__(beverage, custom_cost)

    def get_description(self):
        return self.beverage.get_description() + ", " + "Mocha"

    def cost(self):
        return self.beverage.cost() + self.condiment_cost


class Soy(CondimentDecorator):
    def __init__(self, beverage, custom_cost=0.15):
        super().__init__(beverage, custom_cost)

    def get_description(self):
        return self.beverage.get_description() + ", " + "Soy"

    def cost(self):
        return self.beverage.cost() + self.condiment_cost


class Whip(CondimentDecorator):
    def __init__(self, beverage, custom_cost=0.10):
        super().__init__(beverage, custom_cost)

    def get_description(self):
        return self.beverage.get_description() + ", " + "Whip"

    def cost(self):
        return self.beverage.cost() + self.condiment_cost
class Mocha(CondimentDecorator):
    def __init__(self, beverage, custom_cost=0.20):
        super().__init__(beverage, custom_cost)
class Mocha(CondimentDecorator):
    def __init__(self, beverage, custom_cost=0.20):
        super().__init__(beverage, custom_cost)
```

```
class Milk(CondimentDecorator):
    def __init__(self, beverage, custom_cost=0.10):
        super().__init__(beverage, custom_cost)
class Milk(CondimentDecorator):
    def __init__(self, beverage, custom_cost=0.10):
        super().__init__(beverage, custom_cost)

    def get_description(self):
        return self.beverage.get_description() + ", " + "Whip"
    def get_description(self):
        return self.beverage.get_description() + ", " + "Whip"
    def get_description(self):
        return self.beverage.get_description() + ", " + "Soy"
    def get_description(self):
        return self.beverage.get_description() + ", " + "Soy"
    def cost(self):
        return self.beverage.cost() + self.condiment_cost
    def cost(self):
        return self.beverage.cost() + self.condiment_cost
class Soy(CondimentDecorator):
    def __init__(self, beverage, custom_cost=0.15):
        super().__init__(beverage, custom_cost)
class Soy(CondimentDecorator):
    def __init__(self, beverage, custom_cost=0.15):
        super().__init__(beverage, custom_cost)

    def get_description(self):
        return self.beverage.get_description() + ", " + "Milk"
    def get_description(self):
        return self.beverage.get_description() + ", " + "Milk"

    def get_description(self):
        return self.beverage.get_description() + ", " + "Mocha"
    def get_description(self):
        return self.beverage.get_description() + ", " + "Mocha"
class Whip(CondimentDecorator):
    def __init__(self, beverage, custom_cost=0.10):
        super().__init__(beverage, custom_cost)
class Whip(CondimentDecorator):
    def __init__(self, beverage, custom_cost=0.10):
        super().__init__(beverage, custom_cost)
```

Step 5 – (1 Mark) Update the example usage.

```
# Example usage
beverage = DarkRoast()
beverage = Mocha(beverage)  # Add Mocha with default cost
beverage = Whip(beverage)  # Add Whip with default cost
beverage = Soy(beverage, 0.12)  # Add Soy with a custom cost

print(beverage.get_description())  # Output: Dark Roast Coffee, Mocha, Whip, Soy
print(beverage.cost())  # Output: 1.41 (0.99 base + 0.20 mocha + 0.10 whip + 0.12 soy)
```

```
print(beverage.get_description())
print(beverage.get_description())

beverage = Soy(beverage, 0.12)
beverage = Soy(beverage, 0.12)
beverage = Mocha(beverage)
beverage = Mocha(beverage)

print(beverage.cost())
print(beverage.cost())
beverage = DarkRoast()
beverage = DarkRoast()
beverage = Whip(beverage)
beverage = Whip(beverage)
```

COMP 3522
OBJECT ORIENTED PROGRAMMING 2

Final Exam
December 2024

Full Name	
Signature	
Student ID	
Set	

Instructions:

- This is a closed-book exam. Two cheat sheets are allowed.
- Headphones/ear buds are OK.
- This midterm is 90 minutes long.
- Add the answers for T/F and MCQ questions to the Answer Sheet in Page 1.
- The exam will include the following components:
    - True/False Questions: 5 questions, each worth 1 mark, totaling 5 marks.
    - Multiple-Choice Questions: 15 questions, each worth 2 marks, totaling 30 marks.
    - Draw a UML Diagram: 1 question, worth 4 marks.
    - What is the Output: 1 question, worth 4 marks.
    - Guided Coding: 1 question with two parts, worth 8 marks.

  Total Marks: 51, scaled to account for 30% of the course grade.

- Good luck. You're awesome, you've got this.

Final grade:

True/False Questions
Question 1
The @staticmethod decorator allows a method to access class-level variables.
FALSE

Question 2

The Dependency Inversion Principle (DIP) suggests that both high-level and low-level modules should depend on abstractions, not concrete implementations, which promotes loose coupling and flexibility in the codebase
TRUE

Question 3
Decorator and Proxy have similar structures, but very different intents.
TRUE

Use the following code to answer the following two questions

```
def safe_divide(x, y):
    try:
        return x / y
    except ZeroDivisionError:
        return None  # Or you might want to raise a custom exception

import unittest


class TestExceptionHandling(unittest.TestCase):
    def test_divide_by_zero(self):
        with self.assertRaises(ZeroDivisionError):
            safe_divide(5, 0)

    def test_safe_division(self):
        result = safe_divide(10, 2)
        self.assertIsNone(result)  # Assuming you return None on error

# Run the tests
unittest.main()
```

Question 4
The test_divide_by_zero unit test will pass
FALSE
Question 5
The test_safe_division unit test will pass
FALSE

Multiple Choice Questions
Choose the best answer for the following questions.
Question 6
What is the typical role of the base handler class in the Chain of Responsibility pattern?
A. To define a subscription mechanism for request notifications.
B. To forward requests to the next handler by default.
C. To act as a factory for creating handlers.
D. To store the client's request data persistently.

Question 7
What is a common drawback of the Strategy pattern?
A. It introduces tight coupling between the context and strategy objects.
B. It requires the client to know and select the appropriate strategy.
C. It makes it difficult to extend the behavior of an object.
D. It enforces a static behavior for all context objects.

Question 8
How does the Decorator pattern fundamentally differ from the Proxy pattern in terms of their structure and purpose?

A. Proxy is based on inheritance, while Decorator relies on composition.
B. Decorator manages the lifecycle of its service object, while Proxy does not.
C. Proxy typically controls access or permissions, while Decorator enhances functionality without modifying the interface.
D. Both Decorator and Proxy are interchangeable patterns used for encapsulating objects.

Question 9
How does the Mediator pattern differ from the Observer pattern in its primary goal?
A. Mediator focuses on dynamic, one-way communication, while Observer centralizes communication among objects.
B. Mediator aims to reduce mutual dependencies between components, while Observer establishes dynamic one-way communication.
C. Mediator uses composition, while Observer relies on inheritance
D. Mediator always acts as a publisher, whereas Observer requires no publisher.

Question 10
What is a key difference between the State and Strategy patterns?
A. State restricts dependencies between concrete implementations, while Strategy allows interdependencies.
B. Strategy alters behavior by inheritance, while State alters behavior through composition.
C. State allows transitions between states to alter the context's state, while Strategy isolates different behaviors.
D. State and Strategy are functionally identical and differ only in naming conventions.

Question 11
How does the Facade pattern differ from the Mediator pattern in its relationship to subsystems?
A. Facade allows direct communication between components, while Mediator does not.
B. Mediator defines a simplified interface to a subsystem, while Facade eliminates subsystem dependencies.
C. Facade does not introduce new functionality to a subsystem, while Mediator coordinates component interactions
D. Facade increases coupling between subsystems, while Mediator decreases coupling..

Question 12
What does the time.time() function return?

- The current time in milliseconds since the epoch.
- The current time in microseconds since the epoch.
- The current time in seconds since the epoch.
- The current time in nanoseconds since the epoch.

Question 13
What is the output of the following code?

```
class SomeIterator:
    def __init__(self, limit):
        self.current = limit

    def __iter__(self):
        return self

    def __next__(self):
        if self.current >= 1:
            result = self.current**2
            self.current -= 2
            return result
        else:
            raise StopIteration

for x in SomeIterator(5):
    print(x)
```

Options:

Option A.
25
16
9
4
1

Option B.
25
9
1

Option C.
1
4
9
16
25

Option D.
1
9
25

Question 14
What is the output of the following code?

```
class Coffee:
    def cost(self):
        return 5
```

```
class MilkDecorator:
    def __init__(self, coffee):
        self._coffee = coffee

    def cost(self):
        return self._coffee.cost() + 2


class SugarDecorator:
    def __init__(self, coffee):
        self._coffee = coffee

    def cost(self):
        return self._coffee.cost() + 1


class CaramelDecorator:
    def __init__(self, coffee):
        self._coffee = coffee

    def cost(self):
        return self._coffee.cost() + 3

# Client code
coffee_with_all_decorators = CaramelDecorator(SugarDecorator(MilkDecorator(Coffee())))
print(coffee_with_all_decorators.cost())
print(CaramelDecorator(CaramelDecorator(SugarDecorator(MilkDecorator(Coffee()))))).cost())
```

Options:

A. 10, 13

B. 11, 14

C. 11, 11

D. 8, 11

Question 15

Which of the following UMLs is the actual representation of the following code?

```
from abc import ABC, abstractmethod


class ProxyInterface(ABC):
    @abstractmethod
    def request(self):
        pass
```

```python
class RealSubject(ProxyInterface):
    def request(self):
        return "Handling request by RealSubject"


class CachedProxy(ProxyInterface):
    def __init__(self, subject):
        self._subject = subject
        self._cached_response = None

    def request(self):
        if not self._cached_response:
            print("[CachedProxy] Cache is empty. Fetching from RealSubject...")
            self._cached_response = self._subject.request()
        else:
            print("[CachedProxy] Returning cached response.")
        return self._cached_response


class ProtectionProxy(ProxyInterface):
    def __init__(self, subject, user_role):
        self._subject = subject
        self._user_role = user_role

    def request(self):
        print(
            f"[ProtectionProxy] Verifying access for role: {self._user_role}"
        )
        if self._user_role == "Admin":
            return self._subject.request()
        else:
            return "[ProtectionProxy] Access Denied."


class LoggingProxy(ProxyInterface):
    def __init__(self, subject):
        self._subject = subject

    def request(self):
        print("[LoggingProxy] Logging the request.")
        return self._subject.request()


# Client code
real_subject = RealSubject()
```

```
print("Using Cached Proxy:")
cached_proxy = CachedProxy(real_subject)
print(cached_proxy.request())  # First call, fetches from RealSubject
print(cached_proxy.request())  # Second call, returns cached response

print("\nUsing Protection Proxy:")
admin_proxy = ProtectionProxy(real_subject, "Admin")
user_proxy = ProtectionProxy(real_subject, "User")
print(admin_proxy.request())  # Access granted for Admin
print(user_proxy.request())  # Access denied for User

print("\nUsing Logging Proxy:")
logging_proxy = LoggingProxy(real_subject)
print(logging_proxy.request())  # Logs the request and delegates to RealSubject
```

Options:

A. Option A

B. Option B

C. Option C

Question 16

Which of the following implementations is faithful to the following UML diagram?

Options:

Option A:

```
from abc import ABC, abstractmethod


class TrafficLightState(ABC):
    @abstractmethod
    def next(self, traffic_light):
        pass

class RedState(TrafficLightState):
    def next(self, traffic_light):
        print("Red light -> Switching to green.")
        traffic_light.change_state(GreenState())

class YellowState(TrafficLightState):
    def next(self, traffic_light):
        print("Yellow light -> Switching to red.")
        self.next_state = RedState()
        traffic_light.change_state(self.next_state)
```

```
class GreenState(TrafficLightState):
    def next(self, traffic_light):
        print("Green light -> Switching to yellow.")
        self.next_state = YellowState()
        traffic_light.change_state(self.next_state)


class TrafficLight:
    def __init__(self):
        self.state = RedState()

    def change_state(self, state):
        self.state = state

    def switch(self):
        self.state.next(self)
```

Option B:

```
from abc import ABC, abstractmethod


class TrafficLightState(ABC):
    @abstractmethod
    def next(self, traffic_light):
        pass

class RedState(TrafficLightState):
    def next(self, traffic_light):
        print("Red light -> Switching to green.")
        traffic_light.change_state(GreenState())


class YellowState(TrafficLightState):
    def next(self, traffic_light):
        print("Yellow light -> Switching to red.")
        traffic_light.change_state(RedState())


class GreenState(TrafficLightState):
    def next(self, traffic_light):
        print("Green light -> Switching to yellow.")
        traffic_light.change_state(YellowState())


class TrafficLight:
    def __init__(self):
        self.state = RedState()
```

```
    def change_state(self, state):
        self.state = state

    def switch(self):
        self.state.next(self)
```

Question 17
What is the total time taken to run the following code?

```
import asyncio
import time


class Subject:
    def __init__(self):
        self._observers = []
        self.news_data = {
            "sports": ["Game 1 Highlights", "Player Transfers", "Match Schedules"],
            "tech": ["New AI Breakthrough", "Latest Gadgets", "Tech Company Earnings"],
            "politics": ["Election Results", "Policy Changes", "Government Scandals"],
        }

    def attach(self, observer):
        if observer not in self._observers:
            self._observers.append(observer)

    def detach(self, observer):
        if observer in self._observers:
            self._observers.remove(observer)

    async def notify(self, category):
        for observer in self._observers:
            await observer.update(self, category)

    async def get_latest_news(self, category):
        # Simulate a slight delay in fetching news data
        await asyncio.sleep(0.1)
        return self.news_data.get(category)


class Observer:
    async def update(self, subject, category):
        # Simulate time to receive the category update
        await asyncio.sleep(0.5)
        print(f"Observer received category update: {category}")
        news_items = await subject.get_latest_news(category)
        print(f"Fetching news in category '{category}': {news_items}")
```

```
async def main():
    start_time = time.perf_counter()

    subject = Subject()
    observer_a = Observer()
    observer_b = Observer()
    observer_c = Observer()

    subject.attach(observer_a)
    subject.attach(observer_b)

    task_notify1 = asyncio.create_task(subject.notify("sports"))
    task_notify2 = asyncio.create_task(subject.notify("tech"))


    await task_notify1;
    await task_notify2;
    await subject.notify("politics")

    end_time = time.perf_counter()
    print(f"Total time: {end_time - start_time:.2f} seconds")


if __name__ == "__main__":
    asyncio.run(main())
```

Options:
A. 1.2 seconds
B. 2.4 seconds
C. 3.6 seconds
D. 4.8 seconds

Questions 18
What is the primary purpose of the Builder Design Pattern?
A. To reduce the number of subclasses needed for different configurations.
B. To make object creation faster.
C. To eliminate the need for any constructor and use setters instead.
D. To directly instantiate complex objects with default values.

Questions 19
In the Builder Pattern implementation, what is the role of the Director class?
A. To define the blueprint of the product being built.
B. To orchestrate the construction steps for different configurations.
D. To implement the methods for building specific parts of the product.
D. To instantiate the final object directly.

Questions 20

What is a disadvantage of using the Builder Pattern?
A. Reduced flexibility in object creation.
B. Increased complexity due to additional classes like the builder and director.
C. The need to rewrite the entire product class.
D. Dependency on subclass hierarchies for different configurations.

Draw a UML
Complete the following UML diagram of the Builder design pattern. Do not include the methods in the diagram. Only include the class names and relationships.
Your Answer:
You may use the following blocks. You only need to connect them using the proper relationships.

You may also use the followng example that uses the Builder Design Pattern:

```
class House:
    def __init__(self):
        self.walls = None
        self.roof = None
        self.floor = None
        self.door = None
        self.windows = None
        self.backyard = None
        self.heating_system = None
        self.plumbing = None
        self.electrical_wiring = None

    def __str__(self):
        return (f"House: walls={self.walls}, roof={self.roof}, floor={self.floor}, "
                f"door={self.door}, windows={self.windows}, backyard={self.backyard}, "
                f"heating_system={self.heating_system}, plumbing={self.plumbing}, "
                f"electrical_wiring={self.electrical_wiring}")


class HouseBuilder:
    def build_walls(self):
        pass

    def build_roof(self):
        pass

    def build_floor(self):
        pass

    def install_door(self):
        pass

    def fit_windows(self):
```

```
        pass

    def add_backyard(self):
        pass

    def add_heating_system(self):
        pass

    def add_plumbing(self):
        pass

    def add_electrical_wiring(self):
        pass

    def get_result(self):
        pass


class ConcreteHouseBuilder(HouseBuilder):
    def __init__(self):
        self.house = House()

    def build_walls(self):
        self.house.walls = "Brick walls"
        return self

    def build_roof(self):
        self.house.roof = "Tile roof"
        return self

    def build_floor(self):
        self.house.floor = "Wooden floor"
        return self

    def install_door(self):
        self.house.door = "Wooden door"
        return self

    def fit_windows(self):
        self.house.windows = "Glass windows"
        return self

    def add_backyard(self):
        self.house.backyard = "Small backyard"
        return self

    def add_heating_system(self):
```

```
        self.house.heating_system = "Central heating system"
        return self

    def add_plumbing(self):
        self.house.plumbing = "Modern plumbing"
        return self

    def add_electrical_wiring(self):
        self.house.electrical_wiring = "Standard electrical wiring"
        return self

    def get_result(self):
        return self.house


class HouseDirector:
    def __init__(self, builder):
        self.builder = builder

    def construct_minimal_house(self):
        return (self.builder
                .build_walls()
                .build_roof()
                .build_floor()
                .install_door()
                .fit_windows()
                .get_result())

    def construct_luxury_house(self):
        return (self.builder
                .build_walls()
                .build_roof()
                .build_floor()
                .install_door()
                .fit_windows()
                .add_backyard()
                .add_heating_system()
                .add_plumbing()
                .add_electrical_wiring()
                .get_result())


if __name__ == "__main__":
    builder = ConcreteHouseBuilder()

    director = HouseDirector(builder)
```

```
    # Construct a minimal house
    minimal_house = director.construct_minimal_house()
    print("Minimal House:")
    print(minimal_house)

    # Construct a luxury house
    luxury_house = director.construct_luxury_house()
    print("\nLuxury House:")
    print(luxury_house)
```

Solution:

What is the output

Question

What is the output of the following code? Only fill the following boxes with the output.

```
class Order:
    def __init__(self, items, total_amount, payment_status, shipping_status):
        self.items = items
        self.total_amount = total_amount
        self.payment_status = payment_status
        self.shipping_status = shipping_status


class OrderHandler:
    def __init__(self, next_handler=None):
        self.next_handler = next_handler

    def handle_order(self, order):
        if self.next_handler:
            return self.next_handler.handle_order(order)


class ValidationHandler(OrderHandler):
    def handle_order(self, order):
        if order.items and order.total_amount > 0:
            print("ValidationHandler: Order is valid.")
            super().handle_order(order)
        else:
            print("ValidationHandler: Order is invalid and cannot proceed.")


class AuthorizationHandler(OrderHandler):
    def handle_order(self, order):
        if order.payment_status == "authorized":
            print("AuthorizationHandler: Payment is authorized.")
            super().handle_order(order)
        else:
```

```
        print("AuthorizationHandler: Payment not authorized. Order cannot proceed.")


class DiscountHandler(OrderHandler):
    def handle_order(self, order):
        if order.total_amount > 1000:
            discount = order.total_amount * 0.1
            order.total_amount -= discount
            print(
                f"DiscountHandler: Applied discount of ${discount:.2f}. New total is $
{order.total_amount:.2f}.")
        else:
            print("DiscountHandler: No discount applied.")
        super().handle_order(order)


class PackingHandler(OrderHandler):
    def handle_order(self, order):
        if len(order.items) > 5:
            print("PackingHandler: Bulk order detected. Extra care in packing.")
        else:
            print("PackingHandler: Packing items for shipping.")
        super().handle_order(order)


class ShippingHandler(OrderHandler):
    def handle_order(self, order):
        if order.shipping_status == "pending":
            order.shipping_status = "shipped"
            print("ShippingHandler: Order has been shipped.")
        else:
            print("ShippingHandler: Order already shipped or invalid shipping status.")


# Creating the chain of responsibility
shipping_handler = ShippingHandler()
packing_handler = PackingHandler(shipping_handler)
discount_handler = DiscountHandler(packing_handler)
authorization_handler = AuthorizationHandler(discount_handler)
validation_handler = ValidationHandler(authorization_handler)

# Example order
order = Order(
    items=["Laptop", "Phone", "Tablet", "Mouse", "Keyboard", "Monitor"],
    total_amount=2000,
    payment_status="authorized",
    shipping_status="pending"
```

```
)

# Start the chain with validation
validation_handler.handle_order(order)
```

Your Answer:

A partial output is provide to you. Complete it by filling the blank boxes.

ValidationHandler:
AuthorizationHandler:
DiscountHandler:
PackingHandler:
ShippingHandler:

ValidationHandler: Order is valid.
AuthorizationHandler: Payment is authorized.
DiscountHandler: Applied discount of $200.00. New total is $1800.00.
PackingHandler: Bulk order detected. Extra care in packing.
ShippingHandler: Order has been shipped.

Question - Guided Code Writing

Attached to this questions two code snippets, A and B (pages 20-24). The code snippets are related to the State design pattern. Code Snuppet A, demonstrates a character class that can move, run, jump, attack, stop, finish attacking, and land. Code Snippet B, demonstrates how the State design pattern is used to refactor Code Snippet A.

Part I - Complete a State Diagram

Use Code Snippet A and/or Code Snippet B to add the Attacking state to the following state diagram along with the transitions from and into this state.

Answer:

Note: loop transitions are not shown in the diagram for simplicity.

Solution

Part II - Write Code

Your task is to complete the AttackingState class in Code Snippet B. The remaining classes have already been implemented and are provided and you may look into them as a hint.

To aid your understanding of Code Snippet B, a UML diagram is also included.

You Answer:

```
class AttackingState(CharacterState):
    def move(self, character):
        print("Character cannot walk while attacking.")

    def run(self, character):
        print("Character cannot run while attacking.")
```

```
    def jump(self, character):
        print("Character starts attacking mid-air.")
        character.change_state(JumpingState())

    def attack(self, character):
        print("Character is already attacking.")

    def stop(self, character):
        print("Character cannot stop while attacking.")

    def finish_attack(self, character):
        print("Character finishes the attack.")
        character.change_state(IdleState())
```

Code Snippet A

```
class Character:
    def __init__(self):
        self.state = "Idle"

    def move(self):
        if self.state == "Idle":
            print("Character starts walking.")
            self.state = "Walking"
        elif self.state == "Walking":
            print("Character is already walking.")
        elif self.state == "Running":
            print("Character slows down to walk.")
            self.state = "Walking"
        elif self.state in ["Jumping", "Attacking"]:
            print("Character cannot walk while", self.state.lower() + ".")

    def run(self):
        if self.state == "Idle":
            print("Character starts running.")
            self.state = "Running"
        elif self.state == "Walking":
            print("Character starts running.")
            self.state = "Running"
        elif self.state == "Running":
            print("Character is already running.")
        elif self.state in ["Jumping", "Attacking"]:
            print("Character cannot run while", self.state.lower() + ".")

    def jump(self):
```

```
        if self.state in ["Idle", "Walking", "Running"]:
            print(
                f"Character jumps while {self.state.lower()}." if self.state != "Idle" else "Character
jumps.")
            self.state = "Jumping"
        elif self.state == "Jumping":
            print("Character is already jumping.")
        elif self.state == "Attacking":
            print("Character starts attacking mid-air.")
            self.state = "Jumping"

    def attack(self):
        if self.state in ["Idle", "Walking", "Running"]:
            print(f"Character starts attacking while {self.state.lower()}.")
            self.state = "Attacking"
        elif self.state == "Jumping":
            print("Character starts attacking mid-air.")
            self.state = "Attacking"
        elif self.state == "Attacking":
            print("Character is already attacking.")

    def stop(self):
        if self.state == "Idle":
            print("Character is already idle.")
        elif self.state == "Walking":
            print("Character stops walking.")
            self.state = "Idle"
        elif self.state == "Running":
            print("Character stops running.")
            self.state = "Idle"
        elif self.state in ["Jumping", "Attacking"]:
            print("Character cannot stop while", self.state.lower() + ".")

    def finish_attack(self):
        if self.state == "Attacking":
            print("Character finishes the attack.")
            self.state = "Idle"
        else:
            print("Character is not attacking.")

    def land(self):
        if self.state == "Jumping":
            print("Character lands on the ground.")
            self.state = "Idle"
        else:
            print("Character is not jumping.")
```

```
if __name__ == "__main__":
    character = Character()

    # Character transitions
    character.move()          # Idle -> Walking
    character.run()           # Walking -> Running
    character.jump()          # Running -> Jumping
    character.land()          # Jumping -> Idle
    character.attack()        # Idle -> Attacking
    character.finish_attack()  # Attacking -> Idle
```

Code Snippet B

```
from abc import ABC, abstractmethod


class CharacterState(ABC):
    @abstractmethod
    def move(self, character):
        pass

    @abstractmethod
    def run(self, character):
        pass

    @abstractmethod
    def jump(self, character):
        pass

    @abstractmethod
    def attack(self, character):
        pass

    @abstractmethod
    def stop(self, character):
        pass

    @abstractmethod
    def finish_attack(self, character):
        pass


class IdleState(CharacterState):
    def move(self, character):
        print("Character starts walking.")
        character.change_state(WalkingState())
```

```
    def run(self, character):
        print("Character starts running.")
        character.change_state(RunningState())

    def jump(self, character):
        print("Character jumps.")
        character.change_state(JumpingState())

    def attack(self, character):
        print("Character starts attacking.")
        character.change_state(AttackingState())

    def stop(self, character):
        print("Character is already idle.")

    def finish_attack(self, character):
        print("Character is not attacking.")


class WalkingState(CharacterState):
    def move(self, character):
        print("Character is already walking.")

    def run(self, character):
        print("Character starts running.")
        character.change_state(RunningState())

    def jump(self, character):
        print("Character jumps while walking.")
        character.change_state(JumpingState())

    def attack(self, character):
        print("Character starts attacking while walking.")
        character.change_state(AttackingState())

    def stop(self, character):
        print("Character stops walking.")
        character.change_state(IdleState())

    def finish_attack(self, character):
        print("Character is not attacking.")


class RunningState(CharacterState):
    def move(self, character):
        print("Character slows down to walk.")
```

```
        character.change_state(WalkingState())

    def run(self, character):
        print("Character is already running.")

    def jump(self, character):
        print("Character jumps while running.")
        character.change_state(JumpingState())

    def attack(self, character):
        print("Character starts attacking while running.")
        character.change_state(AttackingState())

    def stop(self, character):
        print("Character stops running.")
        character.change_state(IdleState())

    def finish_attack(self, character):
        print("Character is not attacking.")


class JumpingState(CharacterState):
    def move(self, character):
        print("Character cannot walk while jumping.")

    def run(self, character):
        print("Character cannot run while jumping.")

    def jump(self, character):
        print("Character is already jumping.")

    def attack(self, character):
        print("Character starts attacking mid-air.")
        character.change_state(AttackingState())

    def stop(self, character):
        print("Character cannot stop mid-air.")

    def finish_attack(self, character):
        print("Character is not attacking.")

    def land(self, character):
        print("Character lands on the ground.")
        character.change_state(IdleState())


class AttackingState(CharacterState):
```

```
    pass
    # TODO


class Character:
    def __init__(self):
        self.state = IdleState()

    def change_state(self, new_state):
        self.state = new_state

    def move(self):
        self.state.move(self)

    def run(self):
        self.state.run(self)

    def jump(self):
        self.state.jump(self)

    def attack(self):
        self.state.attack(self)

    def stop(self):
        self.state.stop(self)

    def finish_attack(self):
        self.state.finish_attack(self)

    def land(self):
        if isinstance(self.state, JumpingState):
            self.state.land(self)
        else:
            print("Character is not jumping.")


if __name__ == "__main__":
    character = Character()

    # Character transitions
    character.move()          # Idle -> Walking
    character.run()           # Walking -> Running
    character.jump()          # Running -> Jumping
    character.land()          # Jumping -> Idle
    character.attack()        # Idle -> Attacking
    character.finish_attack()  # Attacking -> Idle
```

The End.